Project Starter py

```
import pandas as pd
import numpy as np
import os
import time
import dotenv
import ast
import json
import re
from sqlalchemy.sql import text
from datetime import datetime, timedelta
from typing import Dict, List, Union, Optional, Tuple
from sqlalchemy import create_engine, Engine

# Create an SQLite database
db_engine = create_engine("sqlite:///munder_difflin.db")

# List containing the different kinds of papers
paper_supplies = [
    # Paper Types (priced per sheet unless specified)
    {"item_name": "A4 paper",                         "category": "paper",
"unit_price": 0.05},
    {"item_name": "Letter-sized paper",              "category": "paper",
"unit_price": 0.06},
    {"item_name": "Cardstock",                        "category": "paper",
"unit_price": 0.15},
    {"item_name": "Colored paper",                    "category": "paper",
"unit_price": 0.10},
    {"item_name": "Glossy paper",                     "category": "paper",
"unit_price": 0.20},
    {"item_name": "Matte paper",                      "category": "paper",
"unit_price": 0.18},
    {"item_name": "Recycled paper",                   "category": "paper",
"unit_price": 0.08},
    {"item_name": "Eco-friendly paper",               "category": "paper",
"unit_price": 0.12},
    {"item_name": "Poster paper",                     "category": "paper",
"unit_price": 0.25},
    {"item_name": "Banner paper",                     "category": "paper",
"unit_price": 0.30},
```

```
    {"item_name": "Kraft paper",                    "category": "paper",
"unit_price": 0.10},
    {"item_name": "Construction paper",             "category": "paper",
"unit_price": 0.07},
    {"item_name": "Wrapping paper",                 "category": "paper",
"unit_price": 0.15},
    {"item_name": "Glitter paper",                  "category": "paper",
"unit_price": 0.22},
    {"item_name": "Decorative paper",               "category": "paper",
"unit_price": 0.18},
    {"item_name": "Letterhead paper",               "category": "paper",
"unit_price": 0.12},
    {"item_name": "Legal-size paper",               "category": "paper",
"unit_price": 0.08},
    {"item_name": "Crepe paper",                    "category": "paper",
"unit_price": 0.05},
    {"item_name": "Photo paper",                    "category": "paper",
"unit_price": 0.25},
    {"item_name": "Uncoated paper",                 "category": "paper",
"unit_price": 0.06},
    {"item_name": "Butcher paper",                  "category": "paper",
"unit_price": 0.10},
    {"item_name": "Heavyweight paper",              "category": "paper",
"unit_price": 0.20},
    {"item_name": "Standard copy paper",            "category": "paper",
"unit_price": 0.04},
    {"item_name": "Bright-colored paper",           "category": "paper",
"unit_price": 0.12},
    {"item_name": "Patterned paper",                "category": "paper",
"unit_price": 0.15},

    # Product Types (priced per unit)
    {"item_name": "Paper plates",                   "category": "product",
"unit_price": 0.10},  # per plate
    {"item_name": "Paper cups",                     "category": "product",
"unit_price": 0.08},  # per cup
    {"item_name": "Paper napkins",                  "category": "product",
"unit_price": 0.02},  # per napkin
    {"item_name": "Disposable cups",                "category": "product",
"unit_price": 0.10},  # per cup
    {"item_name": "Table covers",                   "category": "product",
"unit_price": 1.50},  # per cover
```

```
    {"item_name": "Envelopes",                          "category": "product",
"unit_price": 0.05},  # per envelope
    {"item_name": "Sticky notes",                       "category": "product",
"unit_price": 0.03},  # per sheet
    {"item_name": "Notepads",                           "category": "product",
"unit_price": 2.00},  # per pad
    {"item_name": "Invitation cards",                   "category": "product",
"unit_price": 0.50},  # per card
    {"item_name": "Flyers",                             "category": "product",
"unit_price": 0.15},  # per flyer
    {"item_name": "Party streamers",                    "category": "product",
"unit_price": 0.05},  # per roll
    {"item_name": "Decorative adhesive tape (washi tape)", "category": "product",
"unit_price": 0.20},  # per roll
    {"item_name": "Paper party bags",                   "category": "product",
"unit_price": 0.25},  # per bag
    {"item_name": "Name tags with lanyards",           "category": "product",
"unit_price": 0.75},  # per tag
    {"item_name": "Presentation folders",              "category": "product",
"unit_price": 0.50},  # per folder

    # Large-format items (priced per unit)
    {"item_name": "Large poster paper (24x36 inches)", "category": "large_format",
"unit_price": 1.00},
    {"item_name": "Rolls of banner paper (36-inch width)", "category": "large_format",
"unit_price": 2.50},

    # Specialty papers
    {"item_name": "100 lb cover stock",                "category": "specialty",
"unit_price": 0.50},
    {"item_name": "80 lb text paper",                  "category": "specialty",
"unit_price": 0.40},
    {"item_name": "250 gsm cardstock",                 "category": "specialty",
"unit_price": 0.30},
    {"item_name": "220 gsm poster paper",              "category": "specialty",
"unit_price": 0.35},
]

# Given below are some utility functions you can use to implement your multi-agent
system
```

```python
def generate_sample_inventory(paper_supplies: list, coverage: float = 0.4, seed: int = 137) -> pd.DataFrame:
    """
    Generate inventory for exactly a specified percentage of items from the full paper supply list.

    This function randomly selects exactly `coverage` × N items from the `paper_supplies` list,
    and assigns each selected item:
    - a random stock quantity between 200 and 800,
    - a minimum stock level between 50 and 150.

    The random seed ensures reproducibility of selection and stock levels.

    Args:
        paper_supplies (list): A list of dictionaries, each representing a paper item with
                               keys 'item_name', 'category', and 'unit_price'.
        coverage (float, optional): Fraction of items to include in the inventory (default is 0.4, or 40%).
        seed (int, optional): Random seed for reproducibility (default is 137).

    Returns:
        pd.DataFrame: A DataFrame with the selected items and assigned inventory values, including:
                      - item_name
                      - category
                      - unit_price
                      - current_stock
                      - min_stock_level
    """
    # Ensure reproducible random output
    np.random.seed(seed)

    # Calculate number of items to include based on coverage
    num_items = int(len(paper_supplies) * coverage)

    # Randomly select item indices without replacement
    selected_indices = np.random.choice(
        range(len(paper_supplies)),
        size=num_items,
        replace=False
```

```
    )

    # Extract selected items from paper_supplies list
    selected_items = [paper_supplies[i] for i in selected_indices]

    # Construct inventory records
    inventory = []
    for item in selected_items:
        inventory.append({
            "item_name": item["item_name"],
            "category": item["category"],
            "unit_price": item["unit_price"],
            "current_stock": np.random.randint(200, 800),  # Realistic stock range
            "min_stock_level": np.random.randint(50, 150)  # Reasonable threshold for
reordering
        })

    # Return inventory as a pandas DataFrame
    return pd.DataFrame(inventory)

def init_database(db_engine: Engine, seed: int = 137) -> Engine:
    """
    Set up the Munder Difflin database with all required tables and initial records.

    This function performs the following tasks:
    - Creates the 'transactions' table for logging stock orders and sales
    - Loads customer inquiries from 'quote_requests.csv' into a 'quote_requests' table
    - Loads previous quotes from 'quotes.csv' into a 'quotes' table, extracting useful
metadata
    - Generates a random subset of paper inventory using `generate_sample_inventory`
    - Inserts initial financial records including available cash and starting stock
levels

    Args:
        db_engine (Engine): A SQLAlchemy engine connected to the SQLite database.
        seed (int, optional): A random seed used to control reproducibility of
inventory stock levels.
                              Default is 137.

    Returns:
        Engine: The same SQLAlchemy engine, after initializing all necessary tables and
records.
```

```
    Raises:
        Exception: If an error occurs during setup, the exception is printed and
raised.
    """
    try:
        # ----------------------------
        # 1. Create an empty 'transactions' table schema
        # ----------------------------
        transactions_schema = pd.DataFrame({
            "id": [],
            "item_name": [],
            "transaction_type": [],  # 'stock_orders' or 'sales'
            "units": [],             # Quantity involved
            "price": [],             # Total price for the transaction
            "transaction_date": [],  # ISO-formatted date
        })
        transactions_schema.to_sql("transactions", db_engine, if_exists="replace",
index=False)

        # Set a consistent starting date
        initial_date = datetime(2025, 1, 1).isoformat()

        # ----------------------------
        # 2. Load and initialize 'quote_requests' table
        # ----------------------------
        quote_requests_df = pd.read_csv("quote_requests.csv")
        quote_requests_df["id"] = range(1, len(quote_requests_df) + 1)
        quote_requests_df.to_sql("quote_requests", db_engine, if_exists="replace",
index=False)

        # ----------------------------
        # 3. Load and transform 'quotes' table
        # ----------------------------
        quotes_df = pd.read_csv("quotes.csv")
        quotes_df["request_id"] = range(1, len(quotes_df) + 1)
        quotes_df["order_date"] = initial_date

        # Unpack metadata fields (job_type, order_size, event_type) if present
        if "request_metadata" in quotes_df.columns:
            quotes_df["request_metadata"] = quotes_df["request_metadata"].apply(
                lambda x: ast.literal_eval(x) if isinstance(x, str) else x
```

```
            )
            quotes_df["job_type"] = quotes_df["request_metadata"].apply(lambda x:
x.get("job_type", ""))
            quotes_df["order_size"] = quotes_df["request_metadata"].apply(lambda x:
x.get("order_size", ""))
            quotes_df["event_type"] = quotes_df["request_metadata"].apply(lambda x:
x.get("event_type", ""))

        # Retain only relevant columns
        quotes_df = quotes_df[[
            "request_id",
            "total_amount",
            "quote_explanation",
            "order_date",
            "job_type",
            "order_size",
            "event_type"
        ]]
        quotes_df.to_sql("quotes", db_engine, if_exists="replace", index=False)

        # ----------------------------
        # 4. Generate inventory and seed stock
        # ----------------------------
        inventory_df = generate_sample_inventory(paper_supplies, seed=seed)

        # Seed initial transactions
        initial_transactions = []

        # Add a starting cash balance via a dummy sales transaction
        initial_transactions.append({
            "item_name": None,
            "transaction_type": "sales",
            "units": None,
            "price": 50000.0,
            "transaction_date": initial_date,
        })

        # Add one stock order transaction per inventory item
        for _, item in inventory_df.iterrows():
            initial_transactions.append({
                "item_name": item["item_name"],
                "transaction_type": "stock_orders",
```

```
                "units": item["current_stock"],
                "price": item["current_stock"] * item["unit_price"],
                "transaction_date": initial_date,
            })

        # Commit transactions to database
        pd.DataFrame(initial_transactions).to_sql("transactions", db_engine,
if_exists="append", index=False)

        # Save the inventory reference table
        inventory_df.to_sql("inventory", db_engine, if_exists="replace", index=False)

        return db_engine

    except Exception as e:
        print(f"Error initializing database: {e}")
        raise

def create_transaction(
    item_name: str,
    transaction_type: str,
    quantity: int,
    price: float,
    date: Union[str, datetime],
) -> int:
    """
    This function records a transaction of type 'stock_orders' or 'sales' with a
specified
    item name, quantity, total price, and transaction date into the 'transactions'
table of the database.

    Args:
        item_name (str): The name of the item involved in the transaction.
        transaction_type (str): Either 'stock_orders' or 'sales'.
        quantity (int): Number of units involved in the transaction.
        price (float): Total price of the transaction.
        date (str or datetime): Date of the transaction in ISO 8601 format.

    Returns:
        int: The ID of the newly inserted transaction.

    Raises:
```

```
        ValueError: If `transaction_type` is not 'stock_orders' or 'sales'.
        Exception: For other database or execution errors.
    """
    try:
        # Convert datetime to ISO string if necessary
        date_str = date.isoformat() if isinstance(date, datetime) else date

        # Validate transaction type
        if transaction_type not in {"stock_orders", "sales"}:
            raise ValueError("Transaction type must be 'stock_orders' or 'sales'")

        # Prepare transaction record as a single-row DataFrame
        transaction = pd.DataFrame([{
            "item_name": item_name,
            "transaction_type": transaction_type,
            "units": quantity,
            "price": price,
            "transaction_date": date_str,
        }])

        # Insert the record into the database
        transaction.to_sql("transactions", db_engine, if_exists="append", index=False)

        # Fetch and return the ID of the inserted row
        result = pd.read_sql("SELECT last_insert_rowid() as id", db_engine)
        return int(result.iloc[0]["id"])

    except Exception as e:
        print(f"Error creating transaction: {e}")
        raise

def get_all_inventory(as_of_date: str) -> Dict[str, int]:
    """
    Retrieve a snapshot of available inventory as of a specific date.

    This function calculates the net quantity of each item by summing
    all stock orders and subtracting all sales up to and including the given date.

    Only items with positive stock are included in the result.

    Args:
```

```
        as_of_date (str): ISO-formatted date string (YYYY-MM-DD) representing the
inventory cutoff.

    Returns:
        Dict[str, int]: A dictionary mapping item names to their current stock levels.
    """
    # SQL query to compute stock levels per item as of the given date
    query = """
        SELECT
            item_name,
            SUM(CASE
                WHEN transaction_type = 'stock_orders' THEN units
                WHEN transaction_type = 'sales' THEN -units
                ELSE 0
            END) as stock
        FROM transactions
        WHERE item_name IS NOT NULL
        AND transaction_date <= :as_of_date
        GROUP BY item_name
        HAVING stock > 0
    """

    # Execute the query with the date parameter
    result = pd.read_sql(query, db_engine, params={"as_of_date": as_of_date})

    # Convert the result into a dictionary {item_name: stock}
    return dict(zip(result["item_name"], result["stock"]))

def get_stock_level(item_name: str, as_of_date: Union[str, datetime]) -> pd.DataFrame:
    """
    Retrieve the stock level of a specific item as of a given date.

    This function calculates the net stock by summing all 'stock_orders' and
    subtracting all 'sales' transactions for the specified item up to the given date.

    Args:
        item_name (str): The name of the item to look up.
        as_of_date (str or datetime): The cutoff date (inclusive) for calculating
stock.

    Returns:
```

```
        pd.DataFrame: A single-row DataFrame with columns 'item_name' and
'current_stock'.
    """
    # Convert date to ISO string format if it's a datetime object
    if isinstance(as_of_date, datetime):
        as_of_date = as_of_date.isoformat()

    # SQL query to compute net stock level for the item
    stock_query = """
        SELECT
            item_name,
            COALESCE(SUM(CASE
                WHEN transaction_type = 'stock_orders' THEN units
                WHEN transaction_type = 'sales' THEN -units
                ELSE 0
            END), 0) AS current_stock
        FROM transactions
        WHERE item_name = :item_name
        AND transaction_date <= :as_of_date
    """

    # Execute query and return result as a DataFrame
    return pd.read_sql(
        stock_query,
        db_engine,
        params={"item_name": item_name, "as_of_date": as_of_date},
    )

def get_supplier_delivery_date(input_date_str: str, quantity: int) -> str:
    """
    Estimate the supplier delivery date based on the requested order quantity and a
starting date.

    Delivery lead time increases with order size:
        - ≤10 units: same day
        - 11-100 units: 1 day
        - 101-1000 units: 4 days
        - >1000 units: 7 days

    Args:
        input_date_str (str): The starting date in ISO format (YYYY-MM-DD).
        quantity (int): The number of units in the order.
```

```
    Returns:
        str: Estimated delivery date in ISO format (YYYY-MM-DD).
    """
    # Debug log (comment out in production if needed)
    print(f"FUNC (get_supplier_delivery_date): Calculating for qty {quantity} from date
string '{input_date_str}'")

    # Attempt to parse the input date
    try:
        input_date_dt = datetime.fromisoformat(input_date_str.split("T")[0])
    except (ValueError, TypeError):
        # Fallback to current date on format error
        print(f"WARN (get_supplier_delivery_date): Invalid date format
'{input_date_str}', using today as base.")
        input_date_dt = datetime.now()

    # Determine delivery delay based on quantity
    if quantity <= 10:
        days = 0
    elif quantity <= 100:
        days = 1
    elif quantity <= 1000:
        days = 4
    else:
        days = 7

    # Add delivery days to the starting date
    delivery_date_dt = input_date_dt + timedelta(days=days)

    # Return formatted delivery date
    return delivery_date_dt.strftime("%Y-%m-%d")

def get_cash_balance(as_of_date: Union[str, datetime]) -> float:
    """
    Calculate the current cash balance as of a specified date.

    The balance is computed by subtracting total stock purchase costs ('stock_orders')
    from total revenue ('sales') recorded in the transactions table up to the given
date.

    Args:
```

```
        as_of_date (str or datetime): The cutoff date (inclusive) in ISO format or as a
datetime object.

    Returns:
        float: Net cash balance as of the given date. Returns 0.0 if no transactions
exist or an error occurs.
    """
    try:
        # Convert date to ISO format if it's a datetime object
        if isinstance(as_of_date, datetime):
            as_of_date = as_of_date.isoformat()

        # Query all transactions on or before the specified date
        transactions = pd.read_sql(
            "SELECT * FROM transactions WHERE transaction_date <= :as_of_date",
            db_engine,
            params={"as_of_date": as_of_date},
        )

        # Compute the difference between sales and stock purchases
        if not transactions.empty:
            total_sales = transactions.loc[transactions["transaction_type"] == "sales",
"price"].sum()
            total_purchases = transactions.loc[transactions["transaction_type"] ==
"stock_orders", "price"].sum()
            return float(total_sales - total_purchases)

        return 0.0

    except Exception as e:
        print(f"Error getting cash balance: {e}")
        return 0.0


def generate_financial_report(as_of_date: Union[str, datetime]) -> Dict:
    """
    Generate a complete financial report for the company as of a specific date.

    This includes:
    - Cash balance
    - Inventory valuation
    - Combined asset total
```

```
    - Itemized inventory breakdown
    - Top 5 best-selling products

    Args:
        as_of_date (str or datetime): The date (inclusive) for which to generate the
report.

    Returns:
        Dict: A dictionary containing the financial report fields:
            - 'as_of_date': The date of the report
            - 'cash_balance': Total cash available
            - 'inventory_value': Total value of inventory
            - 'total_assets': Combined cash and inventory value
            - 'inventory_summary': List of items with stock and valuation details
            - 'top_selling_products': List of top 5 products by revenue
    """
    # Normalize date input
    if isinstance(as_of_date, datetime):
        as_of_date = as_of_date.isoformat()

    # Get current cash balance
    cash = get_cash_balance(as_of_date)

    # Get current inventory snapshot
    inventory_df = pd.read_sql("SELECT * FROM inventory", db_engine)
    inventory_value = 0.0
    inventory_summary = []

    # Compute total inventory value and summary by item
    for _, item in inventory_df.iterrows():
        stock_info = get_stock_level(item["item_name"], as_of_date)
        stock = stock_info["current_stock"].iloc[0]
        item_value = stock * item["unit_price"]
        inventory_value += item_value

        inventory_summary.append({
            "item_name": item["item_name"],
            "stock": stock,
            "unit_price": item["unit_price"],
            "value": item_value,
        })
```

```
    # Identify top-selling products by revenue
    top_sales_query = """
        SELECT item_name, SUM(units) as total_units, SUM(price) as total_revenue
        FROM transactions
        WHERE transaction_type = 'sales' AND transaction_date <= :date
        GROUP BY item_name
        ORDER BY total_revenue DESC
        LIMIT 5
    """
    top_sales = pd.read_sql(top_sales_query, db_engine, params={"date": as_of_date})
    top_selling_products = top_sales.to_dict(orient="records")

    return {
        "as_of_date": as_of_date,
        "cash_balance": cash,
        "inventory_value": inventory_value,
        "total_assets": cash + inventory_value,
        "inventory_summary": inventory_summary,
        "top_selling_products": top_selling_products,
    }


def search_quote_history(search_terms: List[str], limit: int = 5) -> List[Dict]:
    """
    Retrieve a list of historical quotes that match any of the provided search terms.

    The function searches both the original customer request (from `quote_requests`)
and
    the explanation for the quote (from `quotes`) for each keyword. Results are sorted
by
    most recent order date and limited by the `limit` parameter.

    Args:
        search_terms (List[str]): List of terms to match against customer requests and
explanations.
        limit (int, optional): Maximum number of quote records to return. Default is 5.

    Returns:
        List[Dict]: A list of matching quotes, each represented as a dictionary with
fields:
            - original_request
            - total_amount
```

```
        - quote_explanation
        - job_type
        - order_size
        - event_type
        - order_date
"""
conditions = []
params = {}

# Build SQL WHERE clause using LIKE filters for each search term
for i, term in enumerate(search_terms):
    param_name = f"term_{i}"
    conditions.append(
        f"(LOWER(qr.response) LIKE :{param_name} OR "
        f"LOWER(q.quote_explanation) LIKE :{param_name})"
    )
    params[param_name] = f"%{term.lower()}%"

# Combine conditions; fallback to always-true if no terms provided
where_clause = " AND ".join(conditions) if conditions else "1=1"

# Final SQL query to join quotes with quote_requests
query = f"""
    SELECT
        qr.response AS original_request,
        q.total_amount,
        q.quote_explanation,
        q.job_type,
        q.order_size,
        q.event_type,
        q.order_date
    FROM quotes q
    JOIN quote_requests qr ON q.request_id = qr.id
    WHERE {where_clause}
    ORDER BY q.order_date DESC
    LIMIT {limit}
"""

# Execute the parameterized query and return ordinary dictionaries.
# pandas keeps this consistent with the other supplied database helpers.
result = pd.read_sql(query, db_engine, params=params)
return result.to_dict(orient="records")
```

```
########################
########################
########################
# YOUR MULTI AGENT STARTS HERE
########################
########################
########################


# The project uses smolagents to give each worker a clear tool boundary.  The
# business decisions inside the tools are deterministic: the LLM framework
# provides the agent architecture, while Python protects inventory and money.
try:
   from smolagents import ToolCallingAgent, tool
   try:
       # Current smolagents releases use OpenAIModel.
       from smolagents import OpenAIModel as CompatibleOpenAIModel
   except ImportError:
       # Earlier releases exposed the same adapter under this name.
       from smolagents import OpenAIServerModel as CompatibleOpenAIModel
except ImportError as exc:
   raise ImportError(
       "smolagents is required. Install it with: pip install smolagents"
   ) from exc


CATALOG = {item["item_name"]: item for item in paper_supplies}


def _catalog_item_from_description(description: str) -> Optional[str]:
   """Map customer wording to one exact item name in the product catalog."""
   value = re.sub(r"[^a-z0-9]+", " ", description.lower()).strip()

   # Specific descriptors must be checked before generic words such as paper.
   alias_rules = [
       (("washi", "adhesive tape"), "Decorative adhesive tape (washi tape)"),
       (("poster board", "poster boards", "24 x 36", "24x36"), "Large poster paper
(24x36 inches)"),
       (("streamer",), "Party streamers"),
       (("paper napkin", "table napkin", "napkin"), "Paper napkins"),
       (("paper cup", "biodegradable cup"), "Paper cups"),
```

```
        (("paper plate", "biodegradable plate"), "Paper plates"),
        (("envelope",), "Envelopes"),
        (("flyer",), "Flyers"),
        (("cardstock", "card stock"), "Cardstock"),
        (("construction paper",), "Construction paper"),
        (("glossy",), "Glossy paper"),
        (("matte",), "Matte paper"),
        (("recycled",), "Recycled paper"),
        (("kraft",), "Kraft paper"),
        (("poster",), "Poster paper"),
        (("colored", "colourful", "colorful", "assorted colors", "various colors"),
"Colored paper"),
        (("printer paper", "printing paper", "a4 paper", "a4 white", "standard paper"),
"A4 paper"),
    ]
    for aliases, item_name in alias_rules:
        if any(alias in value for alias in aliases):
            return item_name
    return None


def _extract_deadline(request_text: str, request_date: str) -> str:
    """Extract a written customer deadline, defaulting to the request date."""
    date_match = re.search(
        r"(?:by|before)\s+([A-Z][a-z]+\s+\d{1,2},\s+\d{4})",
        request_text,
    )
    if not date_match:
        return request_date
    try:
        return datetime.strptime(date_match.group(1), "%B %d, %Y").strftime("%Y-%m-%d")
    except ValueError:
        return request_date


def _extract_requested_items(request_text: str) -> Tuple[List[Dict], List[str]]:
    """Extract quantities and normalize product descriptions from a request."""
    # The supplied evaluator appends this metadata to the natural-language request.
    request_text = request_text.split("(Date of request:", 1)[0]
    # Product dimensions such as 8.5x11 and 24x36 are descriptors, not quantities.
    request_text = re.sub(r"\b\d+(?:\.\d+)?\s*[\"']?\s*x\s*\d+(?:\.\d+)?\b",
"standard-size", request_text, flags=re.IGNORECASE)
```

```
    # Dates occur after these phrases and must not be mistaken for quantities.
    order_text = re.split(
        r"\b(?:please\s+)?(?:deliver(?:ed|y)?|needed\s+by|must\s+be\s+delivered)\b",
        request_text,
        maxsplit=1,
        flags=re.IGNORECASE,
    )[0]
    quantity_pattern = re.compile(
        r"(?P<quantity>\d[\d,]*)\s+"
        r"(?:(?:sheets?|reams?|rolls?|packets?|boxes?)\s+)?"
        r"(?:of\s+)?(?P<description>.*?)"

r"(?=(?:,?\s+(?:and\s+|along\s+with\s+)|,\s*|\n\s*[-*]?\s*)\d[\d,]*\s+|[.\n]|$)",
        re.IGNORECASE,
    )

    combined: Dict[str, int] = {}
    unsupported = []
    for match in quantity_pattern.finditer(order_text):
        quantity = int(match.group("quantity").replace(",", ""))
        description = match.group("description").strip(" -,:;")
        item_name = _catalog_item_from_description(description)
        if item_name:
            combined[item_name] = combined.get(item_name, 0) + quantity
        else:
            unsupported.append(description or "unidentified item")

    items = [
        {"item_name": name, "quantity": quantity}
        for name, quantity in combined.items()
    ]
    return items, unsupported


def _inventory_assessment(
    items: List[Dict], request_date: str, deadline: str
) -> Dict:
    """Check projected stock, affordability and supplier lead times; reorder safely."""
    inventory_snapshot = get_all_inventory(request_date)
    available_cash = get_cash_balance(request_date)
    plans = []
    rejection_reasons = []
```

```
    total_reorder_cost = 0.0

    for item in items:
        item_name = item["item_name"]
        quantity = int(item["quantity"])
        stock_frame = get_stock_level(item_name, deadline)
        projected_stock = int(stock_frame.iloc[0]["current_stock"])
        shortage = max(0, quantity - projected_stock)
        supplier_date = request_date
        reorder_cost = 0.0

        if shortage:
            supplier_date = get_supplier_delivery_date(request_date, shortage)
            reorder_cost = shortage * float(CATALOG[item_name]["unit_price"])
            total_reorder_cost += reorder_cost
            if supplier_date > deadline:
                rejection_reasons.append(
                    f"{item_name}: the earliest supplier arrival is {supplier_date}, "
                    f"after the requested deadline of {deadline}"
                )

        plans.append(
            {
                "item_name": item_name,
                "quantity": quantity,
                "stock_at_deadline": projected_stock,
                "shortage": shortage,
                "supplier_date": supplier_date,
                "reorder_cost": round(reorder_cost, 2),
            }
        )

    if total_reorder_cost > available_cash:
        rejection_reasons.append(
            "the required replenishment cannot be approved within the current
purchasing budget"
        )

    feasible = not rejection_reasons
    if feasible:
        # Mutate stock only after every line has passed feasibility checks.
        for plan in plans:
```

```
            if plan["shortage"]:
                create_transaction(
                    plan["item_name"],
                    "stock_orders",
                    plan["shortage"],
                    plan["reorder_cost"],
                    plan["supplier_date"],
                )

    fulfillment_date = max(
        [request_date] + [plan["supplier_date"] for plan in plans]
    )
    return {
        "feasible": feasible,
        "request_date": request_date,
        "deadline": deadline,
        "fulfillment_date": fulfillment_date,
        "plans": plans,
        "rejection_reasons": rejection_reasons,
        "inventory_items_available_now": len(inventory_snapshot),
    }


@tool
def inventory_assessment_tool(
    items_json: str, request_date: str, deadline: str
) -> str:
    """Assess inventory and create feasible supplier replenishment orders.

    Args:
        items_json: JSON list containing exact item_name and quantity values.
        request_date: Customer request date in YYYY-MM-DD format.
        deadline: Required customer delivery date in YYYY-MM-DD format.

    Returns:
        JSON inventory plan with feasibility, lead times and reorder decisions.
    """
    return json.dumps(_inventory_assessment(json.loads(items_json), request_date,
deadline))


def _generate_quote(items: List[Dict], job: str, need_size: str, event: str) -> Dict:
```

```
    """Price an order using catalog rates, bulk discounting and quote history."""
    search_terms = [term for term in [event, job.split()[-1] if job else ""] if term]
    history = search_quote_history(search_terms[:1], limit=5)
    total_units = sum(int(item["quantity"]) for item in items)
    subtotal = sum(
        int(item["quantity"]) * float(CATALOG[item["item_name"]]["unit_price"]) * 1.35
        for item in items
    )

    if total_units >= 10000:
        discount_rate = 0.12
    elif total_units >= 5000:
        discount_rate = 0.10
    elif total_units >= 1000:
        discount_rate = 0.07
    elif total_units >= 500:
        discount_rate = 0.05
    else:
        # The project README asks every quote to include a bulk incentive.
        discount_rate = 0.02

    total = round(subtotal * (1 - discount_rate), 2)
    return {
        "subtotal": round(subtotal, 2),
        "discount_rate": discount_rate,
        "discount_amount": round(subtotal - total, 2),
        "total": total,
        "total_units": total_units,
        "historical_quotes_consulted": len(history),
        "context": {"job": job, "need_size": need_size, "event": event},
    }


@tool
def quote_generation_tool(
    items_json: str, job: str, need_size: str, event: str
) -> str:
    """Create a competitive catalog quote informed by similar historical quotes.

    Args:
        items_json: JSON list containing exact item_name and quantity values.
        job: Customer job context from the request dataset.
```

```
        need_size: Stated order-size context.
        event: Customer event context.

    Returns:
        JSON quote including total, discount and history-use information.
    """
    return json.dumps(_generate_quote(json.loads(items_json), job, need_size, event))


def _finalize_sale(items: List[Dict], quote: Dict, inventory_plan: Dict) -> Dict:
    """Verify projected stock, record sales, and run a private financial health
check."""
    fulfillment_date = inventory_plan["fulfillment_date"]
    health_report = generate_financial_report(fulfillment_date)
    stock_failures = []

    for item in items:
        stock_frame = get_stock_level(item["item_name"], fulfillment_date)
        stock = int(stock_frame.iloc[0]["current_stock"])
        if stock < int(item["quantity"]):
            stock_failures.append(item["item_name"])

    if stock_failures:
        return {
            "fulfilled": False,
            "reason": "Stock verification failed for: " + ", ".join(stock_failures),
        }

    subtotal = float(quote["subtotal"])
    total = float(quote["total"])
    for item in items:
        line_subtotal = (
            int(item["quantity"])
            * float(CATALOG[item["item_name"]]["unit_price"])
            * 1.35
        )
        line_revenue = total * (line_subtotal / subtotal) if subtotal else 0.0
        create_transaction(
            item["item_name"],
            "sales",
            int(item["quantity"]),
            round(line_revenue, 2),
```

```
            fulfillment_date,
        )

    return {
        "fulfilled": True,
        "fulfillment_date": fulfillment_date,
        "order_total": total,
        # Only a pass/fail health flag is retained; cash/assets are never
customer-facing.
        "internal_health_check": bool(health_report["total_assets"] >= 0),
    }


@tool
def finalize_sale_tool(
    items_json: str, quote_json: str, inventory_plan_json: str
) -> str:
    """Verify stock and finalize a sale by recording database transactions.

    Args:
        items_json: JSON list containing exact item_name and quantity values.
        quote_json: JSON quote produced by the quoting agent.
        inventory_plan_json: JSON feasibility plan produced by the inventory agent.

    Returns:
        JSON fulfillment result containing status, date and customer total.
    """
    return json.dumps(
        _finalize_sale(
            json.loads(items_json),
            json.loads(quote_json),
            json.loads(inventory_plan_json),
        )
    )


def build_agent_team() -> Dict[str, ToolCallingAgent]:
    """Instantiate three smolagents workers with non-overlapping responsibilities."""
    dotenv.load_dotenv()
    api_key = os.getenv("UDACITY_OPENAI_API_KEY")
    if not api_key:
        raise ValueError("UDACITY_OPENAI_API_KEY is missing from the .env file")
```

```
    model = CompatibleOpenAIModel(
        model_id="gpt-4o-mini",
        api_base="https://openai.vocareum.com/v1",
        api_key=api_key,
    )
    inventory_agent = ToolCallingAgent(
        tools=[inventory_assessment_tool],
        model=model,
        name="inventory_agent",
        description="Checks projected stock, supplier timing, cash and safe
reordering.",
        max_steps=4,
    )
    quoting_agent = ToolCallingAgent(
        tools=[quote_generation_tool],
        model=model,
        name="quoting_agent",
        description="Uses quote history and transparent bulk discounts to price
orders.",
        max_steps=4,
    )
    sales_agent = ToolCallingAgent(
        tools=[finalize_sale_tool],
        model=model,
        name="sales_agent",
        description="Performs final stock verification and records fulfilled sales.",
        max_steps=4,
    )
    return {
        "inventory": inventory_agent,
        "quoting": quoting_agent,
        "sales": sales_agent,
    }


def _execute_framework_tool(tool_object, **kwargs) -> str:
    """Execute a smolagents tool while remaining compatible with simple test
doubles."""
    if hasattr(tool_object, "forward"):
        return tool_object.forward(**kwargs)
    return tool_object(**kwargs)
```

```python
class PaperCompanyOrchestratorAgent:
    """Coordinate parsing, inventory, quoting and fulfillment in a fixed safe order."""

    def __init__(self, worker_agents: Dict[str, ToolCallingAgent]):
        self.name = "customer_inquiry_orchestrator"
        self.worker_agents = worker_agents

    def run(
        self,
        customer_request: str,
        job: str,
        need_size: str,
        event: str,
        request_date: str,
    ) -> str:
        """Delegate one request and return an explainable customer-safe response."""
        request_body = customer_request.split("(Date of request:", 1)[0]
        deadline = _extract_deadline(request_body, request_date)
        items, unsupported = _extract_requested_items(customer_request)

        if unsupported or not items:
            missing = ", ".join(unsupported) if unsupported else "the requested
products"
            return (
                "ORDER NOT FULFILLED. We could not match the following item(s) to our "
                f"current paper-products catalog: {missing}. No stock or payment
transaction "
                "was recorded. Please request an available substitute or a revised
order."
            )

        # Delegation 1: Inventory Agent and its inventory/reordering tool.
        inventory_plan = json.loads(
            _execute_framework_tool(
                inventory_assessment_tool,
                items_json=json.dumps(items),
                request_date=request_date,
                deadline=deadline,
            )
        )
```

```
        if not inventory_plan["feasible"]:
            return (
                "ORDER NOT FULFILLED. "
                + "; ".join(inventory_plan["rejection_reasons"])
                + ". No sale was recorded. Please provide a later delivery date or
reduce the quantity."
            )

        # Delegation 2: Quoting Agent and its quote-history/discount tool.
        quote = json.loads(
            _execute_framework_tool(
                quote_generation_tool,
                items_json=json.dumps(items),
                job=job,
                need_size=need_size,
                event=event,
            )
        )

        # Delegation 3: Sales Agent and its transaction/health-check tool.
        sale = json.loads(
            _execute_framework_tool(
                finalize_sale_tool,
                items_json=json.dumps(items),
                quote_json=json.dumps(quote),
                inventory_plan_json=json.dumps(inventory_plan),
            )
        )
        if not sale["fulfilled"]:
            return (
                "ORDER NOT FULFILLED. Final inventory verification was unsuccessful, so
no "
                "sale was completed. Please contact us for an updated availability
check."
            )

        item_summary = ", ".join(
            f"{item['quantity']:,} × {item['item_name']}" for item in items
        )
        discount_text = (
            f" A {quote['discount_rate']:.0%} bulk discount saved "
            f"${quote['discount_amount']:.2f}."
```

```
        )
        history_text = (
            f" Pricing was checked against {quote['historical_quotes_consulted']}
similar "
            "historical quote(s)."
            if quote["historical_quotes_consulted"]
            else " Catalog pricing was used because no close historical quote was
available."
        )
        return (
            f"ORDER FULFILLED. Items: {item_summary}. Quote total:
${quote['total']:.2f}."
            f"{discount_text}{history_text} Delivery is scheduled for "
            f"{sale['fulfillment_date']}, within the requested deadline of {deadline}."
        )


# Run your test scenarios by writing them here. Make sure to keep track of them.

def run_test_scenarios():

    print("Initializing Database...")
    init_database(db_engine)
    try:
        quote_requests_sample = pd.read_csv("quote_requests_sample.csv")
        quote_requests_sample["request_date"] = pd.to_datetime(
            quote_requests_sample["request_date"], format="%m/%d/%y", errors="coerce"
        )
        quote_requests_sample.dropna(subset=["request_date"], inplace=True)
        quote_requests_sample = quote_requests_sample.sort_values("request_date")
    except Exception as e:
        print(f"FATAL: Error loading test data: {e}")
        return

    # Get initial state
    initial_date = quote_requests_sample["request_date"].min().strftime("%Y-%m-%d")
    report = generate_financial_report(initial_date)
    current_cash = report["cash_balance"]
    current_inventory = report["inventory_value"]

    ############
    ############
```

```
############
# INITIALIZE YOUR MULTI AGENT SYSTEM HERE
############
worker_agents = build_agent_team()
multi_agent_system = PaperCompanyOrchestratorAgent(worker_agents)
############
############

results = []
for idx, row in quote_requests_sample.iterrows():
    request_date = row["request_date"].strftime("%Y-%m-%d")

    print(f"\n=== Request {idx+1} ===")
    print(f"Context: {row['job']} organizing {row['event']}")
    print(f"Request Date: {request_date}")
    print(f"Cash Balance: ${current_cash:.2f}")
    print(f"Inventory Value: ${current_inventory:.2f}")

    # Process request
    request_with_date = f"{row['request']} (Date of request: {request_date})"

    ############
    ############
    ############
    # USE YOUR MULTI AGENT SYSTEM TO HANDLE THE REQUEST
    ############
    ############
    ############

    response = multi_agent_system.run(
        customer_request=request_with_date,
        job=str(row["job"]),
        need_size=str(row["need_size"]),
        event=str(row["event"]),
        request_date=request_date,
    )

    # Update state
    report = generate_financial_report(request_date)
    current_cash = report["cash_balance"]
    current_inventory = report["inventory_value"]
```

```
        print(f"Response: {response}")
        print(f"Updated Cash: ${current_cash:.2f}")
        print(f"Updated Inventory: ${current_inventory:.2f}")

        results.append(
            {
                "request_id": idx + 1,
                "request_date": request_date,
                "cash_balance": current_cash,
                "inventory_value": current_inventory,
                "response": response,
            }
        )

        time.sleep(1)

    # Final report
    final_date = quote_requests_sample["request_date"].max().strftime("%Y-%m-%d")
    final_report = generate_financial_report(final_date)
    print("\n===== FINAL FINANCIAL REPORT =====")
    print(f"Final Cash: ${final_report['cash_balance']:.2f}")
    print(f"Final Inventory: ${final_report['inventory_value']:.2f}")

    # Save results
    pd.DataFrame(results).to_csv("test_results.csv", index=False)
    return results


if __name__ == "__main__":
    results = run_test_scenarios()
```